

Programación II

Herencia en C++

Extensión y generalización

Repaso de conceptos

Preguntas de repaso para arrancar la clase:

- ¿Qué es Clasificación? ¿Por qué agrupamos cosas en categorías?
- ¿Qué es Abstracción? ¿Qué detalles ocultamos y cuáles mostramos?
- ¿Qué es Encapsulamiento? ¿Por qué los atributos deben ser private?

Preguntas del tema de hoy:

- ¿Qué es Herencia?

Definición de herencia

✗ Definición obvia

"La herencia es cuando una clase hereda de otra clase"



✓ Definición técnica

"Extender la funcionalidad de una clase que ya existe, sin modificar su código original y, en consecuencia, sin posibilidad de incorporar fallos en los sistemas que ya la usan"

Sintaxis de Herencia en C++

Clase base — Animal.h

```
class Animal {  
private:  
    string _especie;  
    double _peso;  
public:  
    Animal(string e, double p);  
    void mostrarInfoBiologica();  
    void comer();  
};
```

Clase derivada — Mascota.h

```
// ↓ Sintaxis de herencia pública  
class Mascota : public Animal {  
private:  
    string _nombre;  
    string _duenio;  
public:  
    Mascota(string e, double p,  
              string n, string d);  
    void irAlVeterinario();  
};
```



Si se omite public, la herencia es private por defecto → todo falla desde el main.

Las Dos Direcciones de la Herencia

Las dos direcciones de la Herencia

↑ Generalización (Bottom-Up)

```
Mascota
_nombre _edad
```



```
Gato
_nombre
_edad
maúlla()
```

```
Perro
_nombre
_edad
ladra()
```

Se extrae el factor común hacia arriba → se crea la clase base

↓ Extensión (Top-Down)

```
Animal ✓ Ya funciona
_especie _peso
```

↓ *extender sin tocar*

```
Mascota : public Animal
    _nombreDuenio
    _nombreMascota
    irAlVeterinario()
```

Se extiende hacia abajo sin modificar el código original

En ambos casos el resultado es el mismo: una jerarquía Animal → Mascota → {Gato, Perro, Pez...}

El Código y el problema de protected

El problema de protected

① Problema

```
// Desde Gato::envejecer()  
void Gato::envejecer() {  
    _edad--;  
    // ✗ private → no compila  
}
```

② Error común: usar protected

```
// Cambiar Animal.h:  
protected:  
    int _edad; // Ahora visible  
  
// En Gato → compila ... pero:  
    _edad = -10; // ✨ inválido
```

③ La solución correcta

```
// Animal.h sigue igual:  
private:  
    int _edad; // Sin cambios  
  
// En Gato usar el setter:  
    set_edad(nueva); // ✓
```

¿Para qué SÍ sirve protected?

- Para métodos de uso interno que los hijos necesitan, pero que el main NO debe ver.
- Ejemplos: verificarConexion(), buscarEnArray(), validarRango()
- Regla: los ATRIBUTOS siempre private. Solo métodos auxiliares pueden ser protected.

Construcción de clases derivada y base

Construcción de clase derivada y base



"Para que nazca el hijo, primero tiene que construirse el padre"

✗ Incorrecto

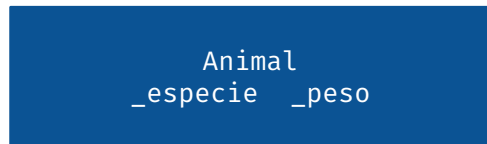
```
Mascota::Mascota(string e, double p,  
                string n, string d) {  
    // ✗ La clase base nunca fue construida  
    _especie = e;  
    _peso    = p;  
    _nombre  = n;  
    _duenio  = d;  
}
```

✓ Correcto — Lista de inicialización

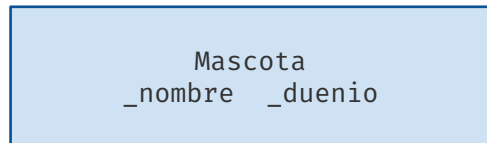
```
Mascota::Mascota(string e, double p,  
                string n, string d)  
    // ↓ Primero construye la clase base  
    : Animal(e, p) {  
    // ✓ Dentro de {}: usar setters  
        set_nombre(n);  
        set_duenio(d);  
    }
```

✓ Dentro de {} del constructor hijo: NO asignar variables directamente. Llamar siempre a los setters para que las validaciones de la clase base se ejecuten.

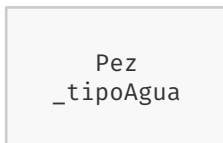
Jerarquía de 3 Niveles — Animal → Mascota → Gato / Pez



↓ : public Animal



↓ : public Mascota



miGato puede llamar a:

```
miGato.mostrarInfoBiologica();  
miGato.mostrarInfoMascota();  
miGato.cazarRatones();
```

Cadena de constructores

```
// Nivel 1 - Animal  
Animal::Animal(string especie, double peso)  
: {  
    set_especie(especie);  
    set_peso(peso);  
}  
  
// Nivel 2 - Mascota proviene de Animal  
Mascota::Mascota(string e, double p, string n, string d)  
: Animal(e, p) {  
    set_nombre(n);  
    set_duenio(d);  
}  
  
// Nivel 3 - Gato fija especie automáticamente proviene de Mascota  
Gato::Gato(double peso, string nombre, string duenio, bool interior)  
: Mascota("Felis catus", peso, nombre, duenio) {  
    _deInterior = interior;  
}
```

Polimorfismo y métodos virtuales

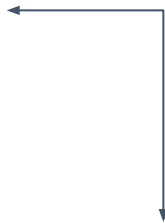
Definición de polimorfismo

✓ Definición técnica

"El polimorfismo es la capacidad de que un mismo método se comporte de manera diferente según el objeto que lo reciba."

Tipos de polimorfismo

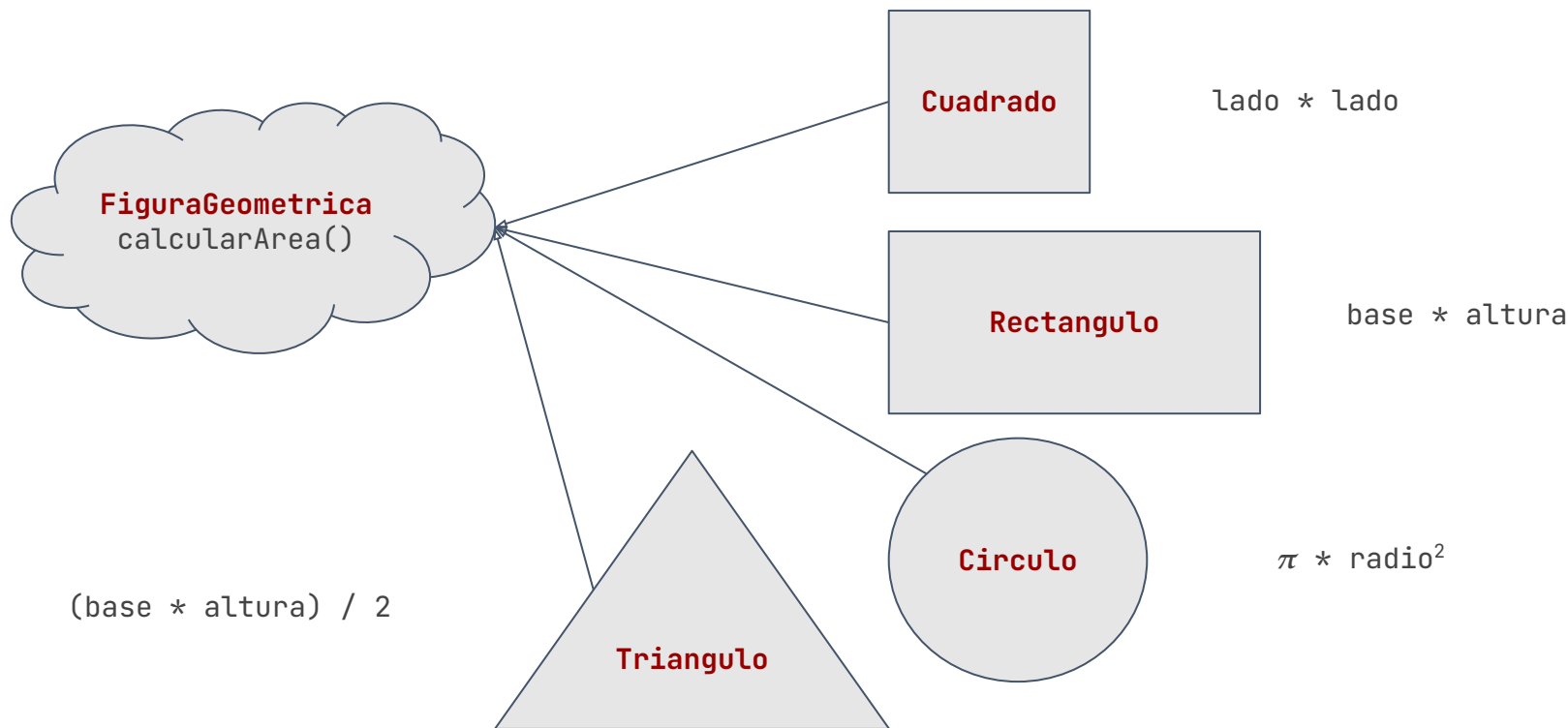
- Polimorfismo en tiempo de compilación (sobrecarga) → Overload
- Polimorfismo en tiempo de ejecución (sobreescripción) → Override



```
class Calculadora {  
public:  
    int sumar(int a, int b) {  
        return a + b;  
    }  
    double sumar(double a, double b) {  
        return a + b;  
    }  
    int sumar(int a, int b, int c) {  
        return a + b + c;  
    }  
};
```

Polimorfismo en tiempo de ejecución

- Pensemos en una figura geométrica. Sin importar cuál sea deberíamos poder calcular su área. Pero cada figura geométrica calcula su área de una manera distinta.



Bonus — Polimorfismo con virtual

Mismo llamado a función → distinto comportamiento según el tipo real del objeto

```
class Figura {
public:
    virtual float calcularArea() {
        return 0.0; // Implementación base
    }
};

class Rectangulo : public Figura {
    float _base, _altura;
public:
    float calcularArea() override {
        return _base * _altura;
    }
};

/// Implementar demás figuras del ejemplo
```

```
// Función polimórfica
void imprimirArea(Figura& f) {
    cout << f.calcularArea() << endl;
}

// En main:
Figura    fig;
Rectangulo rect(5.0, 4.0);
Circulo    circ(3.0);

imprimirArea(fig);    // → 0
imprimirArea(rect);   // → 20
imprimirArea(circ);   // → 28.27
```